Examen Parcial 2 — Programación 1 (F12)

Tormentas Geomagnéticas con la API de NASA / DONKI

Nombre: Katy Ixchely Yaxón Saloj
Carnet: 201944830
Fecha: 22-4-26
Punteo total: 100 puntos

Ejercicio	Tema	Puntos
1a	Construir la URL y realizar la solicitud HTTP	10
1b	Explorar la lista de tormentas con un ciclo `for`	10
2	Calcular el Kp máximo e identificar la tormenta más intensa	20
3	Función `clasificar_kp()` con condicionales	25
4	Función `analizar_tormenta()` que retorna un diccionario	35
	Total	**100**

> **Instrucciones generales**
>
> - Completa cada ejercicio en la celda indicada.
> - Ejecuta las celdas **en orden** de arriba hacia abajo.
> - Puedes consultar tus apuntes y el notebook de clase `XML_JSON_APIs.ipynb`.
> - Entrega el notebook con **todas las celdas ejecutadas** (con output visible).

Contexto: ¿Qué es una tormenta geomagnética?

El Sol emite constantemente partículas cargadas (viento solar). Cuando ocurre una erupción solar intensa, una ola de partículas choca con el campo magnético de la Tierra y provoca una **tormenta geomagnética**.

Estos eventos pueden:

- Generar auroras boreales visibles en latitudes bajas

- Interferir con satélites GPS y comunicaciones de radio
- En casos extremos, dañar redes eléctricas (como el apagón de Quebec en 1989)

Índice Kp — escala de intensidad

El **índice Kp** (0–9) mide la perturbación del campo magnético terrestre:

Kp	Categoría	Descripción
0 – 3	Quieto	Sin tormenta
4	Activo	Perturbación menor
5	**G1** — Menor	Auroras en latitudes altas
6	**G2** — Moderada	Auroras hasta latitud 55°
7	**G3** — Fuerte	Auroras hasta latitud 50°
8	**G4** — Severa	Problemas en redes eléctricas
9	**G5** — Extrema	Apagones posibles, auroras tropicales

La tormenta de mayo 2024 alcanzó **G5** — la más intensa en 20 años.

El endpoint: DONKI / GST

DONKI = Space Weather Database Of Notifications, Knowledge, Information
GST = Geomagnetic Storm

```
GET https://api.nasa.gov/DONKI/GST
```

Parámetro	Tipo	Default	Descripción
`startDate`	YYYY-MM-DD	30 días atrás	Inicio del rango de fechas
`endDate`	YYYY-MM-DD	hoy	Fin del rango de fechas
`api_key`	string	DEMO_KEY	Tu API key de api.nasa.gov

Ejemplo de URL:

```
https://api.nasa.gov/DONKI/GST?startDate=2024-05-01&endDate=2024-05-31&api_key=DEMO_KEY
```

Estructura de la respuesta

La API devuelve una **lista** de eventos. Cada evento es un diccionario con esta estructura:

```
{
  "gstID":    "2024-05-10T17:00:00-GST-001",
  "startTime": "2024-05-10T17:00Z",
  "allKpIndex": [
    { "observedTime": "2024-05-10T21:00Z", "kpIndex": 8.67, "source":
"NOAA" },
    { "observedTime": "2024-05-11T00:00Z", "kpIndex": 9.0,  "source":
"NOAA" }
  ],
  "linkedEvents": [
    { "activityID": "2024-05-08T21:08:00-CME-001" }
  ],
  "link": "https://webtools.ccmc.gsfc.nasa.gov/DONKI/view/GST/..."
}
```

Campos importantes:

- `gstID` — identificador único del evento
- `startTime` — cuándo comenzó la tormenta (formato ISO 8601 UTC)
- `allKpIndex` — lista de mediciones Kp durante la tormenta
- `linkedEvents` — eventos espaciales relacionados (erupciones, CME) — puede ser `None`

Paso 1 — Configuración de la API Key

Obtener tu API key (gratis)

1. Ve a **https://api.nasa.gov**
2. Completa el formulario con tu nombre y correo electrónico
3. Haz clic en **"Signup"**
4. Revisa tu correo — recibirás la key en minutos
5. La key tiene este formato: `TU_API_KEY_AQUI_XXXXXXXXXXXXXXXXXXXXXXXX`

Con tu propia key puedes hacer **1,000 solicitudes por hora** (vs 30 con DEMO_KEY).

Configurar la variable de entorno `NASA_API_KEY`

Una variable de entorno guarda tu key **fuera del código**, para que no quede expuesta en el archivo del notebook ni en el historial de git.

Linux / macOS — temporal (solo la sesión actual)

```
export NASA_API_KEY="tu_key_aqui"
jupyter notebook
```

Linux / macOS — permanente

Agrega esta línea al final de `~/.bashrc` (bash) o `~/.zshrc` (zsh):

```
echo 'export NASA_API_KEY="tu_key_aqui"' >> ~/.bashrc
source ~/.bashrc   # aplicar sin reiniciar
```

Windows — PowerShell (temporal)

```
$env:NASA_API_KEY = "tu_key_aqui"
jupyter notebook
```

Windows — permanente (Panel de control)

1. Busca **"Variables de entorno"** en el menú Inicio
2. Clic en **"Editar las variables de entorno del sistema"**
3. Botón **"Variables de entorno..."**
4. En "Variables de usuario" → **Nueva**
5. Nombre: `NASA_API_KEY` | Valor: tu key
6. Aceptar y **reiniciar** Jupyter

Verificar que está configurada

```
# Linux/macOS
echo $NASA_API_KEY

# Windows PowerShell
echo $env:NASA_API_KEY
```

In [4]:

```python
import os
import requests
import json
import time

# Cargar la API key desde la variable de entorno.
# Si no está configurada, usa DEMO_KEY como respaldo (30 req/hora).
API_KEY = os.getenv("NASA_API_KEY", "").strip() or "DEMO_KEY"

if API_KEY == "DEMO_KEY":
    print("Usando DEMO_KEY — límite: 30 solicitudes/hora")
    print("   Configura NASA_API_KEY para mayor límite (ver instrucciones arriba)."
else:
    print(f"API key cargada: {API_KEY[:4]}{'*' * (len(API_KEY) - 4)}")

print("\nLibrerías importadas correctamente")
```

```
API key cargada: 8Iqa************************************

Librerías importadas correctamente
```

Identificacion del estudiante

Completa tu nombre y carnet en la siguiente celda y **ejecutala antes de comenzar**. Estos datos quedan registrados en el output del notebook junto con la posicion de la ISS en el momento exacto en que realizas el examen.

In [5]:
```
# Completa tus datos
NOMBRE = "Katy Ixchely Yaxón Saloj"   # TU NOMBRE AQUI
CARNET = "201944830"         # TU CARNET AQUI

if NOMBRE == "Nombre Apellido" or CARNET == "202300000":
    raise ValueError("Completa tu nombre y carnet antes de continuar.")

print(f"Estudiante : {NOMBRE}")
print(f"Carnet     : {CARNET}")
```

```
Estudiante : Katy Ixchely Yaxón Saloj
Carnet     : 201944830
```

Ejercicio 1 — Obtener y explorar los datos (20 pts)

Parte a) — Solicitud al endpoint (10 pts)

Se te da la URL base del endpoint. Construye la URL completa usando un f-string e incluye los parámetros `startDate`, `endDate` y `api_key`. Luego realiza la solicitud y verifica que fue exitosa.

¿Por qué verificar el código de estado antes de leer el JSON?

Cuando hacemos una solicitud HTTP el servidor siempre responde con un **código de estado numérico** que indica si todo salió bien o hubo un problema.

El código **200** significa "*OK — la solicitud fue exitosa y el cuerpo de la respuesta contiene los datos pedidos*".

Si el servidor devuelve un código diferente (por ejemplo **503 - Service Unavailable** o **429 - Too Many Requests**), el cuerpo de la respuesta **no contiene JSON válido** — puede estar vacío o contener un mensaje de error en texto plano. Si intentamos llamar `.json()` directamente sin verificar, el programa lanzará un `JSONDecodeError`.

Por eso siempre debemos verificar que `respuesta.status_code == 200` (o equivalentemente `respuesta.ok`) **antes** de intentar parsear la respuesta.

> **Escribe en la celda de código siguiente: ¿qué imprimirías tú para que el usuario sepa qué salió mal cuando el código no es 200?**

In [6]:
```
URL_BASE = "https://api.nasa.gov/DONKI/GST"

# Agrega también los parámetros startDate y endDate:
#   startDate = "2024-05-01"
#   endDate   = "2024-05-31"
# Hint: los parámetros van separados por "&"  →  ?param1=val1&param2=val2

url = f""  # TU CÓDIGO AQUÍ

# Completa la llamada — usa timeout=15
respuesta = ...  # TU CÓDIGO AQUÍ

# Verifica que la respuesta fue exitosa antes de parsear el JSON
# Hint: usa respuesta.ok o compara respuesta.status_code con 200
# Hint: si hay error, imprime el código y el cuerpo — respuesta.text tiene el mensa
if ...:          # TU CÓDIGO AQUÍ — condición de error
    print(...)   # TU CÓDIGO AQUÍ — mensaje informativo
    raise SystemExit("No se pudo obtener la respuesta. Intenta de nuevo.")

tormentas = respuesta.json()
print(f"Solicitud exitosa  — {len(tormentas)} tormentas recibidas")
```

```
Ellipsis
```

```
An exception has occurred, use %tb to see the full traceback.

SystemExit: No se pudo obtener la respuesta. Intenta de nuevo.
```

> ⚠️ **Cuidado con exponer tu API key**
> Puedes descomentar el `print` de la celda siguiente para verificar que la URL se formó correctamente, pero **vuelve a comentarlo antes de guardar y entregar el notebook**. Si lo dejas activo, tu API key quedará visible en el output del archivo.

In [7]:
```
# Descomenta la siguiente línea para verificar que la URL se formó correctamente.
# ⚠ Vuelve a comentarla antes de entregar — el output guarda tu API key en el arch

# print("URL:", url)
```

In [8]:
```
# Importamos librerías necesarias
import os
import requests

# Obtenemos la API key desde la variable de entorno
API_KEY = os.getenv("NASA_API_KEY", "").strip() or "DEMO_KEY"

# Mensaje informativo (opcional, pero útil)
if API_KEY == "DEMO_KEY":
    print("Usando DEMO_KEY (límite bajo)")
else:
    print("API key cargada correctamente")
```

```
API key cargada correctamente
```

In [9]:
```
# Definimos la URL base del endpoint de NASA
URL_BASE = "https://api.nasa.gov/DONKI/GST"

# Definimos la fecha de inicio del rango solicitado
startDate = "2024-05-01"

# Definimos la fecha final del rango solicitado
endDate   = "2024-05-31"

# Construimos la URL completa usando f-string e incluyendo los parámetros requerido
url = f"{URL_BASE}?startDate={startDate}&endDate={endDate}&api_key={API_KEY}"

# Realizamos la solicitud HTTP GET al endpoint con un timeout de 15 segundos
respuesta = requests.get(url, timeout=15)

# Verificamos si la respuesta NO fue exitosa
if not respuesta.ok:
    # Imprimimos el código de error HTTP para informar al usuario
    print(f"Error HTTP: {respuesta.status_code}")

    # Imprimimos el mensaje del servidor para más detalles del error
    print(f"Mensaje del servidor: {respuesta.text}")

    # Terminamos el programa si ocurre un error
    raise SystemExit("No se pudo obtener la respuesta. Intenta de nuevo.")

# Convertimos la respuesta JSON en una estructura de Python (lista de diccionarios)
tormentas = respuesta.json()

# Mostramos cuántas tormentas se recibieron
print(f"Solicitud exitosa  — {len(tormentas)} tormentas recibidas")
```

```
Solicitud exitosa  — 5 tormentas recibidas
```

Parte b) — Explorar la lista de tormentas (10 pts)

Usando un ciclo `for`, imprime cuántas tormentas hubo en el período y el `gstID` y `startTime` de cada una.

In [10]:
```
# Hint: len(tormentas) da el total de eventos
# Hint: cada elemento de tormentas es un dict — accede con tormenta["gstID"], etc.

# TU CÓDIGO AQUÍ
```

In [11]:
```
# Imprimimos el total de tormentas usando len()
print(f"Total de tormentas: {len(tormentas)}\n")

# Recorremos cada tormenta dentro de la lista
for tormenta in tormentas:

    # Imprimimos el ID y el tiempo de inicio de cada tormenta
    print(f"ID: {tormenta['gstID']} | Inicio: {tormenta['startTime']}")
```

```
Total de tormentas: 5

ID: 2024-05-02T15:00:00-GST-001 | Inicio: 2024-05-02T15:00Z
ID: 2024-05-10T15:00:00-GST-001 | Inicio: 2024-05-10T15:00Z
ID: 2024-05-12T21:00:00-GST-001 | Inicio: 2024-05-12T21:00Z
ID: 2024-05-16T06:00:00-GST-001 | Inicio: 2024-05-16T06:00Z
ID: 2024-05-17T18:00:00-GST-001 | Inicio: 2024-05-17T18:00Z
```

Ejercicio 2 — Kp máximo por tormenta (20 pts)

Cada tormenta contiene una lista `allKpIndex` con múltiples mediciones a lo largo del tiempo.

a) (10 pts) Para cada tormenta, encuentra el **valor máximo de Kp** usando un ciclo `for` sobre `allKpIndex`. Imprime el `startTime` y el Kp máximo de cada tormenta.

b) (10 pts) Encuentra e imprime la tormenta **más intensa** del período (la que tuvo el mayor Kp máximo entre todas las tormentas).

In [12]:
```
# ── Parte a) ──────────────────────────────────────────────────────────
# Hint: allKpIndex es una lista de dicts, cada uno con la clave "kpIndex"
# Hint: para encontrar el máximo de una lista puedes usar la función max()
#       o un ciclo que compare con una variable auxiliar (kp_max = 0)

for tormenta in tormentas:
    # TU CÓDIGO AQUÍ — obtener el kp máximo de esta tormenta
    kp_max = ...

    print(f"  {tormenta['startTime']}  Kp máx = {kp_max:.2f}")

# ── Parte b) ──────────────────────────────────────────────────────────
# Hint: necesitas dos variables auxiliares:
#   tormenta_mas_intensa = None
#   kp_global_max = 0
# Recorre todas las tormentas y actualiza estas variables cuando encuentres un Kp m

# TU CÓDIGO AQUÍ
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[12], line 10
      6 for tormenta in tormentas:
      7     # TU CÓDIGO AQUÍ — obtener el kp máximo de esta tormenta
      8     kp_max = ...
      9
---> 10     print(f"  {tormenta['startTime']}  Kp máx = {kp_max:.2f}")
     11
     12 # ── Parte b) ──────────────────────────────────────────────────────────
────
     13 # Hint: necesitas dos variables auxiliares:

TypeError: unsupported format string passed to ellipsis.__format__
```

In [ ]:
```python
# Recorremos cada tormenta en la lista
for tormenta in tormentas:

    # Extraemos la lista de mediciones de Kp
    mediciones = tormenta["allKpIndex"]

    # Creamos una lista con los valores de kpIndex de cada medición
    valores_kp = [m["kpIndex"] for m in mediciones]

    # Calculamos el valor máximo de Kp
    kp_max = max(valores_kp)

    # Imprimimos el tiempo de inicio y el Kp máximo
    print(f"  {tormenta['startTime']}  Kp máx = {kp_max:.2f}")
```

In [ ]:
```python
# Inicializamos variable para guardar la tormenta más intensa
tormenta_mas_intensa = None

# Inicializamos el valor máximo global de Kp
kp_global_max = 0

# Recorremos todas las tormentas
for tormenta in tormentas:

    # Extraemos los valores de Kp
    valores_kp = [m["kpIndex"] for m in tormenta["allKpIndex"]]

    # Calculamos el máximo de esta tormenta
    kp_max = max(valores_kp)

    # Si encontramos un valor mayor al actual, actualizamos
    if kp_max > kp_global_max:
        kp_global_max = kp_max
        tormenta_mas_intensa = tormenta

# Imprimimos la tormenta más intensa encontrada
print("\nTormenta más intensa:")
print(f"Inicio: {tormenta_mas_intensa['startTime']}")
print(f"Kp máximo global: {kp_global_max:.2f}")
```

Ejercicio 3 — Clasificar la severidad con condicionales (25 pts)

a) (15 pts) Escribe una función `clasificar_kp(kp)` que reciba un valor de Kp (float) y retorne una cadena con la categoría según la tabla del contexto:

- Kp < 4 → `"Quieto"`
- Kp == 4 → `"Activo"`
- Kp == 5 → `"G1 - Menor"`
- Kp == 6 → `"G2 - Moderada"`
- Kp == 7 → `"G3 - Fuerte"`
- Kp == 8 → `"G4 - Severa"`
- Kp >= 9 → `"G5 - Extrema"`

> **Nota:** el índice Kp puede ser decimal (ej. 8.67). Usa `int(kp)` para redondear hacia abajo y comparar.

b) (10 pts) Usa tu función para imprimir una tabla con cada tormenta, su Kp máximo y su categoría.

In [13]:
```
def clasificar_kp(kp):
    """Clasifica la intensidad de una tormenta según el índice Kp."""

    # Convertimos el valor a entero para clasificar correctamente
    kp_entero = int(kp)

    # Evaluamos cada rango de la escala
    if kp_entero < 4:
        return "Quieto"
    elif kp_entero == 4:
        return "Activo"
    elif kp_entero == 5:
        return "G1 - Menor"
    elif kp_entero == 6:
        return "G2 - Moderada"
    elif kp_entero == 7:
        return "G3 - Fuerte"
    elif kp_entero == 8:
        return "G4 - Severa"
    else:
        return "G5 - Extrema"
```

In [14]:
```
# ── Parte a) ──────────────────────────────────────────────────────────
# Hint: usa if / elif / else
# Hint: int(8.67) == 8  — esto te permite usar comparaciones exactas con enteros

def clasificar_kp(kp):
    """Clasifica la intensidad de una tormenta según el índice Kp."""
    # TU CÓDIGO AQUÍ
```

```
    pass


# Prueba tu función con estos valores — verifica que los resultados son correctos:
for kp_prueba in [2.0, 4.0, 5.33, 6.67, 7.0, 8.67, 9.0]:
    print(f"  kp={kp_prueba:.2f}  →  {clasificar_kp(kp_prueba)}")

# ── Parte b) ──────────────────────────────────────────────────────────
print(f"\n{'Fecha inicio':<22} {'Kp máx':>7}  {'Categoría'}")
print("-" * 50)

# TU CÓDIGO AQUÍ — recorre tormentas, calcula kp_max, llama clasificar_kp()
```

```
  kp=2.00  →  None
  kp=4.00  →  None
  kp=5.33  →  None
  kp=6.67  →  None
  kp=7.00  →  None
  kp=8.67  →  None
  kp=9.00  →  None

Fecha inicio            Kp máx  Categoría
--------------------------------------------------
```

In [15]:

```
# Imprimimos encabezado de la tabla
print(f"\n{'Fecha inicio':<22} {'Kp máx':>7}  {'Categoría'}")

# Línea separadora
print("-" * 50)

# Recorremos cada tormenta
for tormenta in tormentas:

    # Extraemos valores de Kp
    valores_kp = [m["kpIndex"] for m in tormenta["allKpIndex"]]

    # Calculamos el máximo
    kp_max = max(valores_kp)

    # Clasificamos la tormenta
    categoria = clasificar_kp(kp_max)

    # Imprimimos en formato de tabla
    print(f"{tormenta['startTime']:<22} {kp_max:>7.2f}  {categoria}")
```

```
Fecha inicio            Kp máx  Categoría
--------------------------------------------------
2024-05-02T15:00Z         6.67  None
2024-05-10T15:00Z         9.00  None
2024-05-12T21:00Z         6.33  None
2024-05-16T06:00Z         6.00  None
2024-05-17T18:00Z         6.00  None
```

Ejercicio 4 — Función de análisis completa (35 pts)

Escribe una función `analizar_tormenta(tormenta)` que reciba **un evento** de la lista `tormentas` y retorne un **diccionario** con las siguientes claves:

Clave	Valor esperado
`"id"`	el `gstID` del evento
`"inicio"`	el `startTime`
`"kp_max"`	el valor Kp más alto entre todas las mediciones
`"kp_min"`	el valor Kp más bajo entre todas las mediciones
`"kp_promedio"`	promedio de todos los valores Kp (redondeado a 2 decimales)
`"num_mediciones"`	cuántas mediciones hay en `allKpIndex`
`"categoria"`	resultado de llamar `clasificar_kp(kp_max)`
`"eventos_vinculados"`	número de eventos en `linkedEvents` (0 si es `None`)

Luego aplícala a **todas las tormentas** con un ciclo `for` e imprime un resumen de cada una.

In [16]:
```
def analizar_tormenta(tormenta):
    """
    Analiza un evento de tormenta geomagnética.

    Parámetro:
        tormenta (dict): un elemento de la lista retornada por el endpoint GST.

    Retorna:
        dict con las claves: id, inicio, kp_max, kp_min, kp_promedio,
                             num_mediciones, categoria, eventos_vinculados.
    """
    # Hint: extrae la lista de mediciones primero
    mediciones = tormenta["allKpIndex"]

    # Hint: para kp_promedio, suma todos los kpIndex y divide entre len(mediciones)
    #       usa round(..., 2) para redondear a 2 decimales

    # Hint: linkedEvents puede ser None — usa  (tormenta["linkedEvents"] or [])
    #       para tratar None como lista vacía y poder llamar len() sin error

    # TU CÓDIGO AQUÍ
    resultado = {
        "id":                ...,
        "inicio":            ...,
        "kp_max":            ...,
        "kp_min":            ...,
        "kp_promedio":       ...,
        "num_mediciones":    ...,
        "categoria":         ...,
        "eventos_vinculados": ...,
    }
    return resultado
```

```python
# Aplicar a todas las tormentas e imprimir resumen
for tormenta in tormentas:
    r = analizar_tormenta(tormenta)
    # Hint: accede a cada clave del dict retornado con r["clave"]
    # TU CÓDIGO AQUÍ — imprime los campos de r de forma legible
```

In [17]:
```python
def analizar_tormenta(tormenta):
    """
    Analiza un evento de tormenta geomagnética.
    """

    # Extraemos la lista de mediciones
    mediciones = tormenta["allKpIndex"]

    # Obtenemos los valores de Kp
    valores_kp = [m["kpIndex"] for m in mediciones]

    # Calculamos el valor máximo
    kp_max = max(valores_kp)

    # Calculamos el valor mínimo
    kp_min = min(valores_kp)

    # Calculamos el promedio redondeado a 2 decimales
    kp_promedio = round(sum(valores_kp) / len(valores_kp), 2)

    # Contamos el número de mediciones
    num_mediciones = len(mediciones)

    # Clasificamos la tormenta según su intensidad
    categoria = clasificar_kp(kp_max)

    # Contamos los eventos vinculados (manejo de None)
    eventos_vinculados = len(tormenta["linkedEvents"] or [])

    # Creamos el diccionario resultado
    resultado = {
        "id": tormenta["gstID"],
        "inicio": tormenta["startTime"],
        "kp_max": kp_max,
        "kp_min": kp_min,
        "kp_promedio": kp_promedio,
        "num_mediciones": num_mediciones,
        "categoria": categoria,
        "eventos_vinculados": eventos_vinculados
    }

    # Retornamos el diccionario
    return resultado
```

In [18]:
```python
# Recorremos cada tormenta
for tormenta in tormentas:
```

```
    # Analizamos la tormenta
    r = analizar_tormenta(tormenta)

    # Imprimimos resultados de forma legible
    print("\n--- Tormenta ---")
    print(f"ID: {r['id']}")
    print(f"Inicio: {r['inicio']}")
    print(f"Kp máx: {r['kp_max']}")
    print(f"Kp mín: {r['kp_min']}")
    print(f"Kp promedio: {r['kp_promedio']}")
    print(f"Mediciones: {r['num_mediciones']}")
    print(f"Categoría: {r['categoria']}")
    print(f"Eventos vinculados: {r['eventos_vinculados']}")
```

```
--- Tormenta ---
ID: 2024-05-02T15:00:00-GST-001
Inicio: 2024-05-02T15:00Z
Kp máx: 6.67
Kp mín: 6.67
Kp promedio: 6.67
Mediciones: 2
Categoría: None
Eventos vinculados: 2

--- Tormenta ---
ID: 2024-05-10T15:00:00-GST-001
Inicio: 2024-05-10T15:00Z
Kp máx: 9.0
Kp mín: 6.67
Kp promedio: 8.13
Mediciones: 13
Categoría: None
Eventos vinculados: 6

--- Tormenta ---
ID: 2024-05-12T21:00:00-GST-001
Inicio: 2024-05-12T21:00Z
Kp máx: 6.33
Kp mín: 5.67
Kp promedio: 6.0
Mediciones: 3
Categoría: None
Eventos vinculados: 2

--- Tormenta ---
ID: 2024-05-16T06:00:00-GST-001
Inicio: 2024-05-16T06:00Z
Kp máx: 6.0
Kp mín: 6.0
Kp promedio: 6.0
Mediciones: 1
Categoría: None
Eventos vinculados: 1

--- Tormenta ---
ID: 2024-05-17T18:00:00-GST-001
Inicio: 2024-05-17T18:00Z
Kp máx: 6.0
Kp mín: 6.0
Kp promedio: 6.0
Mediciones: 1
Categoría: None
Eventos vinculados: 2
```

Pregunta final — Conclusiones (incluida en el punteo del Ejercicio 4)

Basandote en los datos que obtuviste a lo largo del examen, escribe en la celda de abajo una conclusion en tus propias palabras. Puedes responder estas preguntas como guia:

- ¿Cual fue la tormenta mas intensa del periodo analizado y que tan severa fue segun la escala Kp?
- ¿Que patrones observas en los datos? (frecuencia, intensidad, eventos vinculados)
- ¿Que impacto podria haber tenido una tormenta de esa magnitud en la vida cotidiana?

Conclusiones:

Durante el análisis de las tormentas geomagnéticas de mayo de 2024 se identificaron un total de 5 eventos. La tormenta más intensa ocurrió el 2024-05-10, alcanzando un valor máximo de Kp = 9.00, lo cual corresponde a una categoría G5 - Extrema.

Se observó que la mayoría de las tormentas presentaron intensidades moderadas (G2), con valores cercanos a Kp 6, lo que indica que los eventos de intensidad media fueron los más frecuentes durante el período analizado. Por otro lado, la tormenta más intensa también fue la que tuvo mayor número de mediciones y eventos vinculados, evidenciando una actividad solar más compleja.

En cuanto a patrones, se puede concluir que a mayor intensidad de la tormenta, mayor es la cantidad de mediciones y eventos relacionados, lo que refleja una evolución más prolongada y significativa del fenómeno.

Una tormenta de magnitud G5 puede tener impactos importantes en la vida cotidiana, como fallas en sistemas GPS, interrupciones en comunicaciones, afectaciones en satélites y posibles problemas en redes eléctricas. Esto resalta la importancia del monitoreo del clima espacial mediante herramientas como la API de NASA.

Sello de entrega

Ejecuta la siguiente celda **como ultimo paso**, justo antes de guardar y subir el notebook.

Realiza una consulta a la API de la ISS para registrar tu posicion geografica aproximada al momento de entregar. Dado que la ISS se desplaza ~30 km cada 4 segundos, dos estudiantes
que entreguen en momentos distintos obtendran coordenadas completamente diferentes.

> Este sello es **unico e irrepetible**: queda vinculado a tu carnet y al instante exacto en que ejecutaste la celda.

In [19]:
```
from datetime import datetime, timezone

# Obtener la posicion actual de la ISS
```

```python
print("Consultando posicion de la ISS...")
try:
    r_iss = requests.get("http://api.open-notify.org/iss-now.json", timeout=8)
    r_iss.raise_for_status()
    iss_data   = r_iss.json()
    iss_lat    = float(iss_data["iss_position"]["latitude"])
    iss_lon    = float(iss_data["iss_position"]["longitude"])
    iss_ts     = iss_data["timestamp"]
    iss_hora   = datetime.fromtimestamp(iss_ts, tz=timezone.utc).strftime("%Y-%m-%d
    iss_ok     = True
except Exception as e:
    iss_lat, iss_lon, iss_hora = 0.0, 0.0, "no disponible"
    iss_ok = False
    print(f"  Advertencia: no se pudo obtener la posicion ISS ({e})")

# Timestamp local del momento de entrega
ts_entrega = datetime.now(tz=timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")

# Imprimir el sello
print()
print("=" * 55)
print("           SELLO DE ENTREGA -- PARCIAL 2")
print("=" * 55)
print(f"  Estudiante   : {NOMBRE}")
print(f"  Carnet       : {CARNET}")
print(f"  Fecha/Hora   : {ts_entrega}")
print(f"  ISS Latitud  : {iss_lat:+.4f} deg")
print(f"  ISS Longitud : {iss_lon:+.4f} deg")
print(f"  ISS Hora UTC : {iss_hora}")
print("=" * 55)

if not iss_ok:
    print("  Advertencia: sello generado sin datos de ISS (sin conexion).")
    print("  Guarda el notebook de todas formas.")
```

```
Consultando posicion de la ISS...

=======================================================
           SELLO DE ENTREGA -- PARCIAL 2
=======================================================
  Estudiante   : Katy Ixchely Yaxón Saloj
  Carnet       : 201944830
  Fecha/Hora   : 2026-04-22 05:05:23 UTC
  ISS Latitud  : +45.5170 deg
  ISS Longitud : -15.3553 deg
  ISS Hora UTC : 2026-04-22 05:03:53 UTC
=======================================================
```

Entrega

1. Ejecuta la celda **Sello de entrega** (arriba)
2. Guarda el notebook: **Archivo** → **Guardar** (o `Ctrl+S`)
3. Verifica que **todas las celdas tienen output** (ejecuta: **Kernel** → **Restart & Run All**)

4. Sube el archivo a Github y copia el link en el campo de entrega en la U Virtual

In [ ]:

```

```